

МРС-66 — Добавить статистику и наблюдаемость запусков краулера

Summary

Добавить в `PriceCrawler` полноценную агрегированную статистику запусков для основных рабочих сценариев:

```
catalog-refresh  
collect-prices  
run-all
```

После выполнения задачи каждый запуск должен сохранять в базе структурированную статистику:

- сколько URL обнаружено;
- сколько записей принято в каталог;
- сколько товаров добавлено;
- сколько обновлено;
- сколько реактивировано;
- сколько деактивировано;
- сколько товаров выбрано для сбора цен;
- сколько поставлено в очередь;
- сколько успешно обработано;
- сколько завершилось retry;
- сколько завершилось dead;
- сколько сохранено snapshots;
- сколько создано новых products;
- сколько возникло ошибок;
- сколько времени заняли отдельные этапы и весь запуск.

Статистика должна быть доступна:

1. в PostgreSQL;
2. в итоговом результате application use case;
3. в структурированных логах;
4. через read-only Web API;
5. в CLI summary Worker.

Главная цель:

```
по одному run_id должно быть понятно,  
что именно сделал запуск,  
сколько данных обработал,
```

где потерял элементы
и сколько времени занял каждый этап.

Context

Репозиторий:

`https://github.com/bodomus/PriceCrawler`

Базовая ветка:

`MPC-65`

Создать рабочую ветку:

`Codex/MPC-66`

Dependencies

Задача зависит от завершённых:

MPC-61 – постоянный `product_catalog`
MPC-62 – `RefreshProductCatalogUseCase`
MPC-63 – `CollectProductPricesUseCase` + `oldest-first`
MPC-64 – `lifecycle active/inactive/reactivated`
MPC-65 – `Worker commands`

До начала реализации проверить наличие:

`IRefreshProductCatalogUseCase`
`ICollectProductPricesUseCase`

и `Worker`-команд:

`catalog-refresh`
`collect-prices`
`run-all`

Также должны существовать:

- `crawler_run`;
- `ingestion_run`;
- `product_catalog_refresh`;
- `price_collect_queue`;
- `product`;
- `price_snapshot`;
- `crawl_error` или эквивалентная таблица ошибок.

Если зависимости отсутствуют, не создавать параллельные временные механизмы.
Зафиксировать блокирующую зависимость.

Problem

Сейчас результат запуска частично описывается:

- статусом;
- строковым `note`;
- отдельными логами;
- состояниями queue;
- записями ошибок.

Этого недостаточно для production-эксплуатации.

Основные проблемы:

1. строка `note` не является структурированными данными;
 2. невозможно надёжно построить отчёт по запускам;
 3. невозможно сравнить производительность двух запусков;
 4. неизвестно, на каком этапе потерялись URL;
 5. отсутствует единая статистика catalog refresh и price collection;
 6. невозможно быстро определить:
 7. discovery ничего не нашёл;
 8. upsert отбросил записи;
 9. queue не приняла URL;
 10. extractor завершился ошибками;
 11. snapshot не был создан;
 12. Worker выводит summary, но данные не имеют единого persistence contract;
 13. нет read-only API для просмотра истории запусков;
 14. длительность этапов не сохраняется структурированно.
-

Goal

Добавить единый observability flow:

```
Application use case
  ↓
Run statistics accumulator
  ↓
crawler_run statistics persistence
  ↓
structured logs
  ↓
Worker summary
  ↓
read-only Web API
```

Статистика должна быть разделена по типам запусков:

```
catalog-refresh
price-collection
run-all
legacy
```

`run-all` является process-level командой и может агрегировать два отдельных `crawler_run`, но не должен искусственно смешивать статистику catalog refresh и price collection в одну DB-запись.

Required architecture

Соблюдать существующие слои:

```
Domain
Application
Infrastructure
Worker
Web
Tests
```

Domain

Domain должен содержать:

- immutable stats models;
- read repository interfaces;

- enums или стабильные строковые контракты;
- application-independent статистические структуры.

Domain не должен зависеть от:

- PostgreSQL;
- Npgsql;
- EF Core;
- Application;
- Infrastructure;
- Worker;
- Web.

Application

Application должен:

- собирать counters;
- измерять этапы;
- передавать статистику repository;
- формировать result models;
- не содержать SQL;
- не выполнять ad hoc запросы к таблицам статистики.

Infrastructure

Infrastructure должна содержать:

- schema changes;
- DB routines;
- repository implementations;
- mapping;
- агрегирующие SQL queries для read API.

Worker

Worker должен:

- выводить итоговый summary;
- не вычислять статистику заново из базы;
- использовать result use case.

Web

Web должен:

- предоставлять read-only endpoints;
- не изменять состояние запусков;
- не содержать SQL.

Scope

1. Расширить `crawler_run`

Предпочтительно расширить существующую таблицу `crawler_run`, а не создавать отдельную таблицу со статистикой верхнего уровня.

Добавить колонки:

```
run_type          varchar(50) not null default 'legacy',
discovery_source  varchar(50),

started_at        timestampz not null,
finished_at       timestampz,
duration_ms       bigint,

discovered_count  integer not null default 0,
accepted_count    integer not null default 0,
inserted_count    integer not null default 0,
updated_count     integer not null default 0,
reactivated_count integer not null default 0,
deactivated_count integer not null default 0,

selected_count    integer not null default 0,
enqueued_count    integer not null default 0,
succeeded_count   integer not null default 0,
retry_count       integer not null default 0,
dead_count        integer not null default 0,
failed_count      integer not null default 0,

products_created_count integer not null default 0,
products_updated_count integer not null default 0,
snapshots_created_count integer not null default 0,
errors_created_count  integer not null default 0,

error_code        varchar(100),
error_message     varchar(1000),

created_at        timestampz not null default now(),
updated_at        timestampz not null default now()
```

Если часть колонок уже существует, не создавать дубликаты.

Допускается использовать отдельную таблицу:

```
crawler_run_stats
```

только при наличии чёткого архитектурного обоснования.

Предпочтительный вариант — структурированные поля в `crawler_run`.

2. Constraints

Добавить check constraints:

```
duration_ms >= 0
discovered_count >= 0
accepted_count >= 0
inserted_count >= 0
updated_count >= 0
reactivated_count >= 0
deactivated_count >= 0
selected_count >= 0
enqueued_count >= 0
succeeded_count >= 0
retry_count >= 0
dead_count >= 0
failed_count >= 0
products_created_count >= 0
products_updated_count >= 0
snapshots_created_count >= 0
errors_created_count >= 0
```

Добавить check:

```
finished_at is null
or finished_at >= started_at
```

Допустимые `run_type`:

```
catalog-refresh
price-collection
legacy
```

Если проект использует свободные строковые source values, допускается не вводить DB enum, но добавить application constants.

3. Индексы

Добавить:

```
create index if not exists ix_crawler_run_started_at
on crawler_run(started_at desc);
```

```
create index if not exists ix_crawler_run_run_type_started_at
on crawler_run(run_type, started_at desc);
```

```
create index if not exists ix_crawler_run_status_started_at
on crawler_run(status, started_at desc);
```

Не добавлять лишние индексы без использования.

Domain statistics models

4. Добавить CrawlerRunStatistics

Добавить immutable model:

```
public sealed record CrawlerRunStatistics(
    int DiscoveredCount,
    int AcceptedCount,
    int InsertedCount,
    int UpdatedCount,
    int ReactivatedCount,
    int DeactivatedCount,
    int SelectedCount,
    int EnqueuedCount,
    int SucceededCount,
    int RetryCount,
    int DeadCount,
    int FailedCount,
    int ProductsCreatedCount,
    int ProductsUpdatedCount,
    int SnapshotsCreatedCount,
    int ErrorsCreatedCount);
```

Все значения должны быть неотрицательными.

Допускается разделить на:

```
CatalogRefreshStatistics
PriceCollectionStatistics
```

и иметь общий верхнеуровневый тип.

Предпочтительно не создавать один большой mutable DTO.

5. Добавить CrawlerRunStageTiming

```
public sealed record CrawlerRunStageTiming(  
    string Stage,  
    long DurationMs);
```

Допустимые этапы:

```
discovery  
catalog-upsert  
catalog-deactivation  
catalog-selection  
queue-enqueue  
queue-processing  
run-finalization
```

Строковые значения вынести в constants:

```
CrawlerRunStages
```

Не использовать произвольные строки по всему коду.

6. Добавить read model запуска

Добавить:

```
public sealed record CrawlerRunDetails(  
    long Id,  
    string RunType,  
    string Source,  
    string? DiscoverySource,  
    string Status,  
    DateTimeOffset StartedAtUtc,  
    DateTimeOffset? FinishedAtUtc,  
    long? DurationMs,  
    CrawlerRunStatistics Statistics,  
    IReadOnlyList<CrawlerRunStageTiming> StageTimings,  
    string? ErrorCode,  
    string? ErrorMessage,  
    string? Note);
```

7. Добавить summary model

Добавить облегчённую модель:

```
public sealed record CrawlerRunSummary(  
    long Id,  
    string RunType,  
    string Source,  
    string Status,  
    DateTimeOffset StartedAtUtc,  
    DateTimeOffset? FinishedAtUtc,  
    long? DurationMs,  
    int PrimaryCount,  
    int SucceededCount,  
    int FailedCount,  
    string? ErrorCode);
```

PrimaryCount :

- для catalog refresh — `accepted_count` ;
- для price collection — `selected_count` ;
- для legacy — определить по текущей семантике либо использовать `0` .

Предпочтительно не вводить неоднозначный `PrimaryCount` , если API может возвращать полные именованные counters. Допускается вернуть расширенный summary.

Stage timing persistence

8. Добавить таблицу `crawler_run_stage`

Добавить:

```
create table if not exists crawler_run_stage  
(  
    id          bigint generated always as identity primary key,  
    run_id      bigint not null references crawler_run(id) on delete cascade,  
    stage       varchar(100) not null,  
    started_at  timestampz not null,  
    finished_at timestampz not null,  
    duration_ms bigint not null,  
    item_count  integer,  
    created_at  timestampz not null default now()  
);
```

Constraints:

```
duration_ms >= 0
finished_at >= started_at
item_count is null or item_count >= 0
```

Unique constraint:

```
unique(run_id, stage)
```

Если один этап может выполняться несколько раз, использовать:

```
stage_sequence integer not null
unique(run_id, stage, stage_sequence)
```

Для текущей задачи предпочтительно один агрегированный этап одного типа на run.

9. Индекс stage table

Добавить:

```
create index if not exists ix_crawler_run_stage_run_id
on crawler_run_stage(run_id);
```

Repository contracts

10. Расширить ICrawlerRunRepository

Не ломать существующие вызовы без необходимости.

Предпочтительно добавить новые методы:

```
Task<long> StartAsync(
    string runType,
    string source,
    string? discoverySource,
    CancellationToken ct);
```

```
Task CompleteAsync(
    long runId,
    RunStatus status,
    CrawlerRunStatistics statistics,
```

```
IReadOnlyCollection<CrawlerRunStageTiming> stageTimings,  
string? note,  
string? errorCode,  
string? errorMessage,  
CancellationToken ct);
```

Старые методы:

```
StartAsync(string source, ...)  
FinishAsync(...)
```

можно сохранить временно для legacy compatibility.

Не создавать несколько несовместимых путей завершения run без чёткого mapping.

11. Добавить read repository

Добавить:

```
public interface ICrawlerRunReadRepository  
{  
    Task<CrawlerRunDetails?> GetByIdAsync(  
        long runId,  
        CancellationToken ct);  
  
    Task<IReadOnlyList<CrawlerRunSummary>> GetRecentAsync(  
        int limit,  
        string? runType,  
        string? status,  
        CancellationToken ct);  
}
```

Добавить:

```
Task<CrawlerRunAggregateStatistics> GetAggregateAsync(  
    DateTimeOffset fromUtc,  
    DateTimeOffset toUtc,  
    string? runType,  
    CancellationToken ct);
```

Если aggregate endpoint выходит за разумный score, он всё равно обязателен в рамках MPC-66, поскольку задача посвящена статистике запусков.

12. Aggregate model

Добавить:

```
public sealed record CrawlerRunAggregateStatistics(  
    DateTimeOffset FromUtc,  
    DateTimeOffset ToUtc,  
    string? RunType,  
    int TotalRuns,  
    int SuccessfulRuns,  
    int FailedRuns,  
    long TotalDurationMs,  
    double AverageDurationMs,  
    long TotalDiscovered,  
    long TotalAccepted,  
    long TotalSelected,  
    long TotalSucceeded,  
    long TotalDead,  
    long TotalSnapshotsCreated,  
    long TotalErrorsCreated);
```

Средние значения не должны приводить к делению на ноль.

DB routines

13. Добавить или обновить routines

Добавить либо изменить:

```
crawler_run_start  
crawler_run_complete  
crawler_run_stage_replace  
crawler_run_get_by_id  
crawler_run_get_recent  
crawler_run_get_aggregate
```

Если текущие routines имеют другие имена, следовать стилю проекта.

14. `crawler_run_start`

Должна принимать:

```
p_run_type
p_source
p_discovery_source
```

и создавать run со статусом:

```
running
```

Counters должны быть нулевыми.

15. `crawler_run_complete`

Должна одним вызовом сохранять:

- status;
- finished_at;
- duration_ms;
- все counters;
- note;
- error code;
- error message.

Обязательное требование:

завершение run и сохранение итоговой статистики не должны выполняться десятками отдельных update-запросов.

Предпочтительно один DB routine call.

16. Stage timings

Сохранение stage timings должно выполняться:

- либо одним `jsonb` batch;
- либо массивами;
- либо в рамках `crawler_run_complete`.

Запрещено выполнять один DB round-trip на каждый stage, если их можно передать batch-ом.

17. Read routines

Read routines должны:

- поддерживать `limit`;

- ограничивать limit безопасным максимумом;
- поддерживать фильтр `run_type`;
- поддерживать фильтр `status`;
- возвращать последние записи по `started_at desc`;
- не использовать dynamic SQL без необходимости.

Максимальный limit:

200

Default:

50

Statistics collection service

18. Добавить application accumulator

Добавить stateless или scoped компонент:

`ICrawlerRunStatisticsCollector`
`CrawlerRunStatisticsCollector`

или:

`CrawlerRunMetricsBuilder`

Предпочтительный вариант — mutable object с ограниченной жизнью внутри одного use case:

```
public sealed class CrawlerRunMetrics
```

Он может содержать методы:

```
SetDiscovered(int count)
SetCatalogUpsert(...)
SetSelection(...)
SetQueueStats(...)
IncrementSnapshotCreated(...)
IncrementProductCreated(...)
IncrementProductUpdated(...)
IncrementErrorCreated(...)
StartStage(string stage)
```

```
FinishStage(string stage, int? itemCount = null)
Build()
```

Требования:

- используется только в пределах одного run;
- не регистрируется Singleton;
- thread-safe для parallel queue processing;
- counters обновляются через `Interlocked` или другой корректный механизм;
- stage names централизованы;
- отрицательные значения невозможны.

19. Timing

Для измерения duration использовать:

```
Stopwatch
```

для elapsed duration.

Для persisted timestamps использовать:

```
DateTimeOffset.UtcNow
```

или существующую clock abstraction.

Не вычислять duration только как разницу wall-clock, если можно использовать monotonic `Stopwatch`.

Catalog refresh integration

20. Обновить `RefreshProductCatalogUseCase`

Use case должен собирать:

```
discovered_count
accepted_count
inserted_count
updated_count
reactivated_count
deactivated_count
```



```
errors_created_count
duration_ms
```

Stage timings:

```
discovery
catalog-upsert
catalog-deactivation
run-finalization
```

Required result consistency

Поля `RefreshProductCatalogResult` должны совпадать с сохранённой DB-статистикой.

Запрещена ситуация:

```
result.InsertedCount = 100
crawler_run.inserted_count = 99
```

Использовать один источник данных — accumulator/result.

21. Catalog safety failures

Если refresh завершён ошибкой из-за safety guard:

```
catalog_refresh_below_minimum
catalog_refresh_ratio_too_low
```

сохранить:

- discovered count;
- accepted count;
- upsert counters, если upsert уже был выполнен;
- deactivated count = 0;
- status = error;
- error code;
- stage timings.

Статистика не должна обнуляться только потому, что финальный статус error.

Price collection integration

22. Обновить `CollectProductPricesUseCase`

Сохранять:

```
selected_count
enqueued_count
succeeded_count
retry_count
dead_count
failed_count
products_created_count
products_updated_count
snapshots_created_count
errors_created_count
duration_ms
```

Stage timings:

```
catalog-selection
queue-enqueue
queue-processing
run-finalization
```

23. Snapshot repository result

Если существующий `StoreObservationAsync` уже возвращает:

```
ProductId
SnapshotId
ProductCreated
```

расширить или использовать имеющиеся данные для counters:

```
products_created_count
products_updated_count
snapshots_created_count
```

Обязательная семантика:

- новый product → `ProductsCreatedCount + 1` ;
- существующий product, обновлённый observation → `ProductsUpdatedCount + 1` ;

- каждый успешно созданный snapshot → `SnapshotsCreatedCount + 1`.

Если snapshot не создаётся из-за deduplication, это должно отражаться фактическим значением.

Не считать successful queue item автоматически равным snapshot created, если DB routine возвращает иной результат.

24. Error counters

`ErrorsCreatedCount` увеличивается только если запись ошибки действительно создана в БД.

`FailedCount` :

число item-level обработок,
которые не завершились success

Итоговая семантика должна быть документирована.

Не смешивать:

queue retry
dead item
crawl_error row

Queue statistics consistency

25. Источник queue stats

Итоговые значения:

succeeded
retry
dead
pending

должны браться из существующего:

`GetRunStatsAsync(runId)`

или эквивалента.

Не вычислять финальные queue counters только из локальных in-memory событий, поскольку Worker может:

- повторять item;
- восстанавливать lease;
- обрабатывать retry;
- пережить частичный сбой.

In-memory accumulator можно использовать для дополнительных counters, но финальные queue status counters должны соответствовать БД.

Failure and cancellation

26. Top-level failure

При unexpected error сохранять доступную частичную статистику:

- stage timings, завершённые до ошибки;
- counters, накопленные до ошибки;
- error code;
- error message;
- duration.

Не завершать run нулевой статистикой, если часть работы уже выполнена.

27. Cancellation

При cancellation:

- сохранить доступную частичную статистику, если возможно;
- status использовать существующий поддерживаемый `error` либо `cancelled`;
- error code:

cancelled

- повторно выбросить `OperationCanceledException`;
 - Worker должен вернуть exit code `12`.
-

28. Failure while persisting statistics

Если основная операция завершилась, но `crawler_run_complete` упала:

- логировать отдельную критическую ошибку;
- не возвращать ложный success;
- command exit code должен быть non-zero;

- не проглатывать persistence failure.
-

Web API

29. Добавить read-only controller

Добавить:

```
GET /api/crawler-runs
GET /api/crawler-runs/{id}
GET /api/crawler-runs/statistics
```

Следовать текущему route style проекта, если он отличается.

30. GET /api/crawler-runs

Query parameters:

```
limit
runType
status
```

Default:

```
limit = 50
```

Maximum:

```
limit = 200
```

Пример:

```
GET /api/crawler-runs?limit=20&runType=price-collection&status=ok
```

Response должен содержать список summary.

31. GET /api/crawler-runs/{id}

Возвращает:

- общие данные run;
- counters;
- stage timings;
- error data;
- note.

Если run отсутствует:

404

Некорректный ID:

400

32. GET /api/crawler-runs/statistics

Query parameters:

from
to
runType

Требования:

- from и to — UTC ISO-8601;
- from < to;
- максимальный диапазон без отдельного разрешения:

365 days

Default range:

последние 30 дней

Response:

```
{  
  "fromUtc": "...",
```

```
"toUtc": "...",
"runType": "price-collection",
"totalRuns": 30,
"successfulRuns": 28,
"failedRuns": 2,
"totalDurationMs": 123456,
"averageDurationMs": 4115.2,
"totalSelected": 150000,
"totalSucceeded": 148500,
"totalDead": 1500,
"totalSnapshotsCreated": 148500,
"totalErrorsCreated": 1700
}
```

33. API constraints

API read-only.

Запрещено добавлять endpoints:

- запуск crawler;
- остановка crawler;
- удаление run;
- изменение status;
- повторный запуск;
- очистка статистики.

Worker output

34. catalog-refresh

CLI summary должен использовать сохранённый result:

```
Command: catalog-refresh
Status: ok
RunId: 125
Duration: 00:18:42
Discovered: 40120
Accepted: 40080
Inserted: 150
Updated: 39900
Reactivated: 30
Deactivated: 5
```

35. collect-prices

```
Command: collect-prices
Status: ok
RunId: 126
Duration: 01:42:16
Selected: 5000
Enqueued: 5000
Succeeded: 4930
Retry: 40
Dead: 30
Products created: 120
Products updated: 4810
Snapshots created: 4930
Errors created: 70
```

36. run-all

Выводить два отдельных summary:

```
Catalog refresh:
  RunId: ...
  Status: ...

Price collection:
  RunId: ...
  Status: ...
```

Не суммировать несопоставимые counters.

Logging

37. Completion logs

Catalog:

```
Catalog refresh completed. RunId={RunId}; DurationMs={DurationMs};
Discovered={Discovered}; Accepted={Accepted}; Inserted={Inserted};
Updated={Updated}; Reactivated={Reactivated}; Deactivated={Deactivated}
```

Price:


```
Price collection completed. RunId={RunId}; DurationMs={DurationMs};
Selected={Selected}; Enqueued={Enqueued}; Succeeded={Succeeded};
Retry={Retry}; Dead={Dead}; SnapshotsCreated={SnapshotsCreated};
ErrorsCreated={ErrorsCreated}
```

38. Stage logs

Допускается логировать завершение этапов:

```
Crawler stage completed. RunId={RunId}; Stage={Stage};
DurationMs={DurationMs}; ItemCount={ItemCount}
```

Не логировать stage start/finish для каждого item.

Documentation

39. README

Добавить раздел:

```
Run statistics
```

Описать:

- какие counters сохраняются;
 - разницу между catalog и price stats;
 - CLI summary;
 - API endpoints;
 - SQL-примеры.
-

40. Architecture

Обновить:

```
docs/architecture.md
```

Добавить flow:

Use case

- statistics collector
- crawler_run
- crawler_run_stage
- logs / CLI / Web API

41. SQL examples

Добавить:

```
select
    id,
    run_type,
    status,
    started_at,
    finished_at,
    duration_ms,
    discovered_count,
    accepted_count,
    selected_count,
    succeeded_count,
    dead_count,
    snapshots_created_count,
    error_code
from crawler_run
order by id desc
limit 50;
```

```
select
    run_id,
    stage,
    duration_ms,
    item_count
from crawler_run_stage
where run_id = :run_id
order by id;
```

```
select
    run_type,
    count(*) as runs,
    avg(duration_ms) as avg_duration_ms,
    sum(succeeded_count) as succeeded,
    sum(dead_count) as dead
from crawler_run
```

```
where started_at >= now() - interval '30 days'  
group by run_type;
```

Tests

Добавить unit, repository integration, API и end-to-end tests.

Unit tests: accumulator

1. Default counters

```
CrawlerRunMetrics_NewInstance_AllCountersAreZero
```

2. Increment counters

```
CrawlerRunMetrics_IncrementOperations_UpdateExpectedCounters
```

3. Thread safety

```
CrawlerRunMetrics_ConcurrentIncrements_AreNotLost
```

Использовать параллельные операции.

4. Negative protection

```
CrawlerRunMetrics_NegativeValues_AreRejectedOrNormalized
```

Предпочтительно rejected.

5. Stage duration

```
CrawlerRunMetrics_CompletedStage_ProducesNonNegativeDuration
```

6. Duplicate stage

```
CrawlerRunMetrics_DuplicateStageCompletion_IsRejected
```

или агрегируется детерминированно.

Unit tests: catalog integration

7. Success counters

RefreshProductCatalogUseCase_Success_PersistsCompleteStatistics

8. Safety failure keeps partial stats

RefreshProductCatalogUseCase_SafetyFailure_PersistsDiscoveryAndUpsertCounters

9. Discovery failure

RefreshProductCatalogUseCase_DiscoveryFailure_PersistsErrorAndDuration

10. Stage timings

RefreshProductCatalogUseCase_Success_PersistsExpectedStages

Unit tests: price integration

11. Success

CollectProductPricesUseCase_Success_PersistsQueueAndSnapshotStatistics

12. Empty selection

CollectProductPricesUseCase_NoDueProducts_PersistsZeroSuccessfulStatistics

13. Retry/dead

CollectProductPricesUseCase_RetryAndDead_PersistsFinalQueueStats

14. Product created/updated counters

CollectProductPricesUseCase_Observations_CountsCreatedAndUpdatedProducts

15. Error rows

```
CollectProductPricesUseCase_CrawlErrors_CountsActuallyPersistedErrors
```

16. Cancellation partial stats

```
CollectProductPricesUseCase_Cancelled_PersistsPartialStatistics
```

PostgreSQL integration tests

17. Complete run persistence

```
CrawlerRunRepository_Complete_PersistsAllCounters
```

18. Stage batch persistence

```
CrawlerRunRepository_Complete_PersistsStagesInSingleBatch
```

Если невозможно доказать single round-trip напрямую, проверить API/mock executor invocation count.

19. Read by ID

```
CrawlerRunReadRepository_GetById_MapsCountersAndStages
```

20. Recent filtering

```
CrawlerRunReadRepository_GetRecent_FiltersByRunTypeAndStatus
```

21. Limit cap

```
CrawlerRunReadRepository_GetRecent_CapsLimitAtMaximum
```

22. Aggregate

```
CrawlerRunReadRepository_GetAggregate_ReturnsCorrectTotals
```

23. Empty aggregate

```
CrawlerRunReadRepository_GetAggregate_EmptyRange_ReturnsZeros
```

24. Constraints

```
CrawlerRun_NegativeCounter_IsRejected
```

25. Stage uniqueness

```
CrawlerRunStage_DuplicateStageForRun_IsRejected
```

26. Schema idempotency

Повторный bootstrap не падает.

API tests

27. List recent

```
GET /api/crawler-runs  
→ 200
```

28. Filter

```
GET /api/crawler-runs?runType=price-collection&status=ok
```

возвращает только подходящие записи.

29. Get details

```
GET /api/crawler-runs/{id}  
→ counters + stages
```

30. Missing ID

```
GET /api/crawler-runs/999999  
→ 404
```

31. Invalid limit

Проверить:

```
limit = 0  
limit = 10000
```

Поведение должно быть детерминированным:

- 0 → 400 либо safe minimum;
- 10000 → cap 200 либо 400.

Предпочтительно model validation с `400`.

32. Aggregate default range

```
GET /api/crawler-runs/statistics
```

использует 30 дней.

33. Invalid date range

```
from >= to  
→ 400
```

34. Too large range

```
range > 365 days  
→ 400
```

End-to-end tests

35. Catalog refresh statistics

Запустить controlled refresh.

Проверить:

```
result counters
=
crawler_run counters
=
API details counters
```

36. Price collection statistics

Запустить controlled price collection.

Проверить:

```
selected = queue rows for run
succeeded = succeeded queue rows
dead = dead queue rows
snapshots created = actual inserted snapshots
errors created = actual error rows
```

37. Failed run

Заставить extractor или repository завершиться ошибкой.

Проверить:

- status error;
- counters до ошибки сохранены;
- error code сохранён;
- duration заполнен;
- API возвращает данные.

Performance requirements

38. No per-item statistics writes

Запрещено обновлять `crawler_run` после каждого товара.

Итоговая статистика сохраняется:

- один раз при завершении;
- допускается редкий progress update только если уже существует отдельный механизм.

MPC-66 не должна создавать 5000 дополнительных update-запросов для 5000 товаров.

39. Stage writes

Stage timings должны сохраняться batch-ом.

40. Read queries

`GET /api/crawler-runs` не должен загружать stage rows для каждого summary.

Избегать N+1.

Details endpoint может отдельным запросом читать stages.

Non-goals

В рамках MPC-66 запрещено реализовывать:

- Grafana;
 - Prometheus exporter;
 - OpenTelemetry collector;
 - Jaeger;
 - distributed tracing;
 - email alerts;
 - Telegram alerts;
 - automatic retry запуска;
 - scheduler;
 - dashboard UI;
 - charts в Web UI;
 - экспорт CSV;
 - очистку старой статистики;
 - retention policy;
 - partitioning таблиц;
 - multi-store analytics;
 - изменение бизнес-логики discovery;
 - изменение oldest-first;
 - изменение deactivation policy;
 - изменение extractor;
 - изменение частоты запуска;
 - запуск Worker через API;
 - изменение Worker command contract;
 - хранение per-product timing;
 - логирование каждого URL.
-

Important design decisions

1. Статистика является частью `crawler_run`

Она не должна существовать только в логах.

2. Result, DB и CLI должны совпадать

Использовать один источник counters.

3. Частичная статистика сохраняется при ошибке

Error status не означает counters = 0.

4. Queue counters берутся из БД

Итоговые состояния queue должны соответствовать фактическим rows.

5. Не писать статистику по одному item

Только агрегаты.

6. Stage timings отдельные

Длительность этапов не хранить внутри строки `note`.

7. API только read-only

Управление Worker через Web не входит в scope.

8. `run-all` не создаёт искусственный объединённый run

Catalog и price collection остаются отдельными runs.

Acceptance criteria

Задача считается выполненной, если:

- [] `crawler_run` содержит структурированные counters;
- [] добавлен `run_type`;
- [] добавлен `duration_ms`;
- [] добавлены error code/message;
- [] добавлена таблица `crawler_run_stage`;
- [] добавлены constraints;

- [] добавлены indexes;
- [] добавлены domain statistics models;
- [] добавлен thread-safe metrics collector;
- [] добавлены stage constants;
- [] RefreshProductCatalogUseCase сохраняет catalog counters;
- [] CollectProductPricesUseCase сохраняет price counters;
- [] partial stats сохраняются при ошибке;
- [] cancellation сохраняет доступные stats;
- [] queue final stats соответствуют БД;
- [] product created/updated counters вычисляются по факту;
- [] snapshot counter вычисляется по факту;
- [] error counter вычисляется по факту записи ошибки;
- [] run completion выполняется одним DB routine call;
- [] stages сохраняются batch-ом;
- [] добавлен ICrawlerRunReadRepository ;
- [] реализован PostgreSQL read repository;
- [] добавлен list API;
- [] добавлен details API;
- [] добавлен aggregate statistics API;
- [] API поддерживает фильтры и limits;
- [] API не содержит write operations;
- [] Worker summary использует result use case;
- [] run-all выводит два отдельных summary;
- [] README обновлён;
- [] architecture documentation обновлена;
- [] добавлены unit tests;
- [] добавлены PostgreSQL integration tests;
- [] добавлены API tests;
- [] добавлены end-to-end consistency tests;
- [] отсутствуют per-item statistics updates;
- [] отсутствует N+1 в list endpoint;
- [] schema bootstrap идиомпотентен;
- [] существующие тесты проходят;
- [] solution собирается без новых warnings;
- [] unrelated code не рефакторился.

Required verification

Codex обязан выполнить:

```
dotnet restore
dotnet build --no-restore
dotnet test --no-build
```

Запустить PostgreSQL integration infrastructure.

Manual verification

1. Catalog refresh

```
dotnet run --project VarPrice.Worker -- catalog-refresh
```

Проверить:

```
select
    id,
    run_type,
    status,
    duration_ms,
    discovered_count,
    accepted_count,
    inserted_count,
    updated_count,
    reactivated_count,
    deactivated_count,
    error_code
from crawler_run
order by id desc
limit 1;
```

2. Catalog stages

```
select
    stage,
    duration_ms,
    item_count
from crawler_run_stage
where run_id = :run_id
order by id;
```

Ожидаемые этапы:

```
discovery
catalog-upsert
catalog-deactivation
run-finalization
```

`catalog-deactivation` может отсутствовать либо иметь нулевой item count, если этап был безопасно пропущен. Поведение документировать.

3. Price collection

Использовать небольшой batch:

```
{
  "Crawler": {
    "MaxProductsPerRun": 10,
    "UseStubProductCardExtractor": true
  }
}
```

Запуск:

```
dotnet run --project VarPrice.Worker -- collect-prices
```

Проверить:

```
select
  id,
  run_type,
  status,
  selected_count,
  enqueued_count,
  succeeded_count,
  retry_count,
  dead_count,
  products_created_count,
  products_updated_count,
  snapshots_created_count,
  errors_created_count,
  duration_ms
from crawler_run
order by id desc
limit 1;
```

4. Сверка с queue

```
select
  status,
  count(*)
from price_collect_queue
```

```
where run_id = :run_id  
group by status;
```

Значения должны совпадать с `crawler_run`.

5. Сверка snapshots

```
select count(*)  
from price_snapshot  
where crawler_run_id = :run_id;
```

Если текущая schema использует другое имя FK, адаптировать запрос.

Результат должен совпадать с:

```
snapshots_created_count
```

6. API recent runs

```
GET /api/crawler-runs?limit=10
```

7. API details

```
GET /api/crawler-runs/{runId}
```

Проверить counters и stages.

8. API aggregate

```
GET /api/crawler-runs/statistics?runType=price-collection
```

Codex final response format

Финальный ответ Codex обязан содержать следующие разделы.

1. Summary

Кратко описать новую систему статистики.

2. Changed files

Перечислить все добавленные и изменённые файлы с назначением.

3. Database changes

Перечислить:

- новые колонки;
- constraints;
- indexes;
- новую таблицу stages;
- новые и изменённые routines.

4. Statistics contract

Привести полный список counters и их точную семантику.

5. Runtime integration

Показать:

```
Catalog refresh
→ metrics
→ crawler_run + stages

Price collection
→ metrics
→ crawler_run + stages
```

6. Consistency guarantees

Подтвердить:

- result = DB counters;
- Worker summary = result;
- API reads DB;
- queue counters соответствуют фактическим queue rows;
- snapshot counter соответствует фактическим inserts.

7. API

Перечислить endpoints, query parameters и ограничения.

8. Performance

Подтвердить:

- нет update statistics на каждый товар;
- stages сохраняются batch-ом;
- list endpoint не имеет N+1.

9. Tests

Указать:

- unit tests;
- PostgreSQL integration tests;
- API tests;
- end-to-end tests;
- фактический результат `dotnet build`;
- фактический результат `dotnet test`.

10. Manual verification evidence

Привести фактические значения:

```
Catalog RunId:
Catalog DurationMs:
Discovered:
Accepted:
Inserted:
Updated:
Reactivated:
Deactivated:

Price RunId:
Price DurationMs:
Selected:
Enqueued:
Succeeded:
Retry:
Dead:
ProductsCreated:
ProductsUpdated:
SnapshotsCreated:
ErrorsCreated:
```


11. Risks and follow-up

Явно указать:

- dashboard UI ещё не реализован;
- external monitoring ещё не подключён;
- retention policy отсутствует;
- alerts отсутствуют;
- scheduler относится к следующему этапу.

Не заявлять об успешном тестировании, сборке или ручной проверке, если они фактически не выполнялись.